

Simple Explanation of Cost Function and Gradient Descent

Main Takeaway: A **cost function** measures how “wrong” a model’s predictions are, and **gradient descent** is a step-by-step method to adjust model parameters to make that cost as small as possible.

1. What Is a Cost Function?

A **cost function** (also called a loss function) assigns a single number to how far off your model’s predictions are from the true values. In simple linear regression, the most common cost function is the **Mean Squared Error (MSE)**:

$$\text{MSE}(m, b) = \frac{1}{n} \sum_{i=1}^n (y_i - (mx_i + b))^2$$

- x_i : input data point
- y_i : true output
- m, b : slope and intercept of the predicted line
- n : number of data points

MSE squares each error $(y_i - \hat{y}_i)$ so positive and negative errors both count positively, then averages them. A **lower MSE** means your line fits the data points better^[1].

2. Visualizing the Cost Function

Imagine plotting the cost (vertical axis) against different choices of (m, b) . For a single parameter case, this curve looks like a **U-shaped parabola**: high cost on both sides and lowest cost at the bottom.

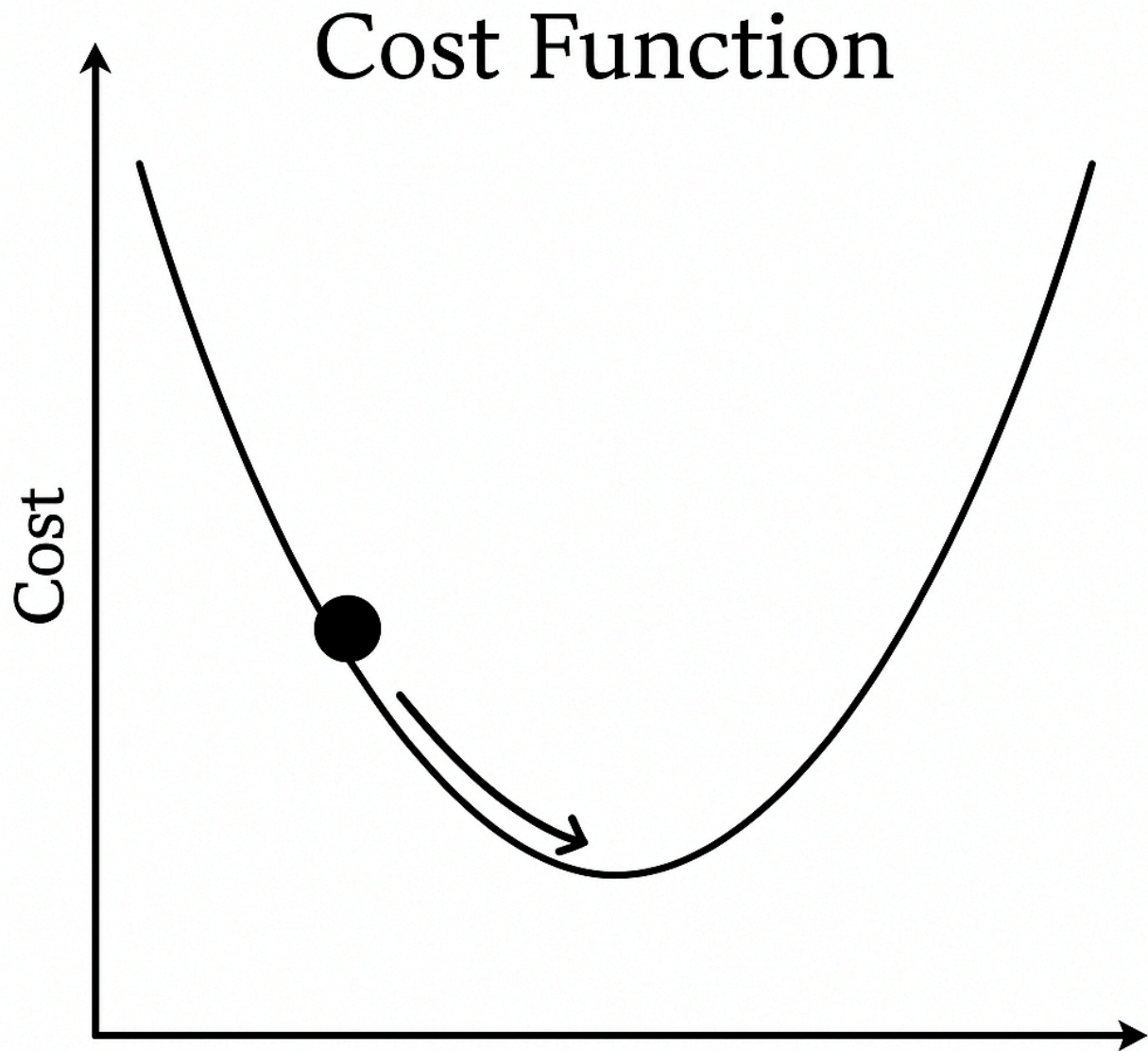


Illustration of a cost function curve with a gradient descent step

Illustration of a cost function curve with a gradient descent step

3. How Gradient Descent Works

Gradient descent finds the bottom of that U-shaped curve by taking small steps downhill:

1. **Initialize:** Start with an arbitrary guess for the parameters (m, b) .
2. **Compute Gradient:** Calculate how much the cost would change if you tweaked each parameter. Mathematically, this is the partial derivative of the cost function with respect to each parameter:

$$\frac{\partial \text{MSE}}{\partial m}, \quad \frac{\partial \text{MSE}}{\partial b}$$

3. **Update Parameters:** Move parameters in the direction that decreases cost:

$$m \leftarrow m - \alpha \frac{\partial \text{MSE}}{\partial m}, \quad b \leftarrow b - \alpha \frac{\partial \text{MSE}}{\partial b}$$

- α is the **learning rate**, a small constant controlling step size.

4. **Repeat:** Recompute gradient with the new (m, b) and step again until the cost change becomes negligible.

Each update slides you down the slope of the parabola toward the minimum. If the learning rate is too large, you may overshoot; too small, and convergence is slow^[2].

4. Key Intuition

- **Gradient** = slope of the cost curve at your current point; tells you which way is “downhill.”
- **Learning rate (α)** = how big of a step to take each time.
- **Convergence** = when further updates no longer significantly lower the cost.

Together, these ensure you systematically refine your model parameters to minimize prediction error.

**

1. <https://www.kdnuggets.com/2020/05/5-concepts-gradient-descent-cost-function.html>

2. <https://www.ibm.com/think/topics/gradient-descent>